

Project Summary: AgentOS – An Operating System-Inspired Runtime for Autonomous AI Agents

High-Level Vision

AgentOS is not a replacement for Linux or Windows. Instead, it is an operating system-inspired control plane/runtime that runs on top of Linux and manages AI agents the way an operating system manages processes.

The central idea is that AI agents become first-class system objects.

Today, most agent frameworks treat agents as Python objects inside an application. AgentOS instead treats them as independent processes managed by a central runtime.

Applications should describe what they want accomplished, while AgentOS decides how agents execute, communicate, wait, recover, and use resources.

Core Philosophy

Traditional operating systems abstract hardware.

AgentOS abstracts intelligence.

Linux manages:

- CPU
- Memory
- Files
- Networking
- Processes

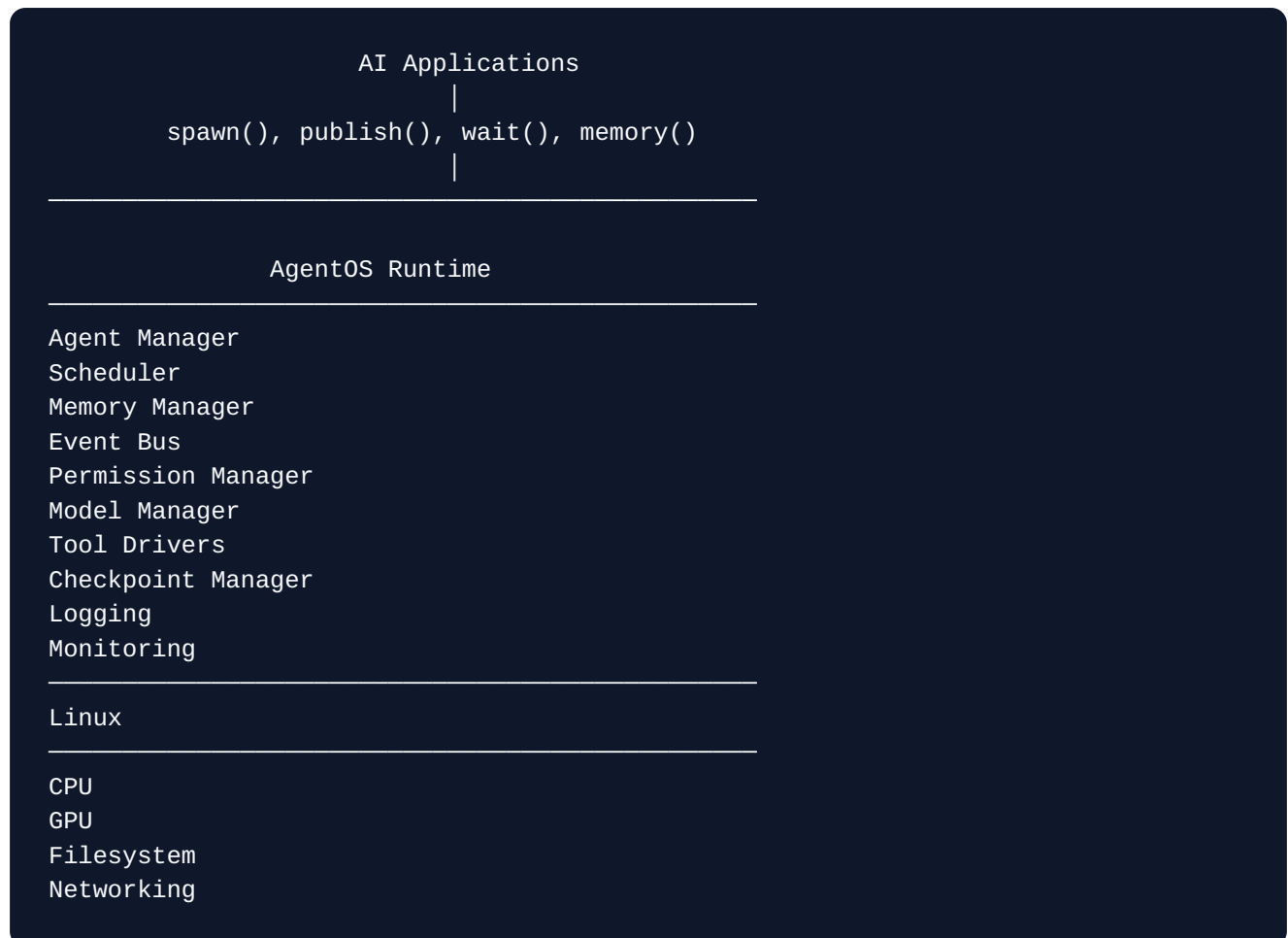
AgentOS manages:

- Agents
- LLMs
- Context
- Memory
- Tool execution
- Human approvals
- Agent communication
- Cost

- Checkpoints
- Dependencies

Linux remains responsible for hardware. AgentOS becomes responsible for intelligent computation.

Overall Architecture



Agent Process Abstraction

Every AI agent is treated as a process.

Instead of

```
research_agent.run()
```

developers create

```
kernel.spawn(ResearchAgent())
```

Every running agent has metadata similar to a Linux process.

Example:

- PID: 17

- Name: Research Agent
- Parent: Planner
- Children: 5
- Status: Running
- Priority: High
- Model: GPT-5
- Permissions: Browser, Python
- Memory: 512 KB
- Checkpoint: #31
- Waiting On: None

The runtime owns every agent. Applications no longer directly orchestrate them.

Agent Lifecycle

Agents transition between states. Possible states include:

- Ready
- Running
- Waiting
- Sleeping
- Blocked
- Finished
- Failed
- Checkpointing
- Suspended

Example:

Planner
↓
Creates Research Agent
↓
Running
↓
Waiting for Search
↓
Running
↓
Waiting for Human Approval
↓

Running

↓

Finished

Scheduler

The scheduler is responsible for deciding which agent executes next. Instead of applications manually calling agents in sequence, developers simply submit work.

Example:

```
kernel.spawn(ProjectPlanner())
```

The scheduler decides:

- execution order
- priorities
- dependency satisfaction
- retries
- starvation prevention
- fairness
- concurrency

Scheduling is based on agent state rather than CPU instructions. Possible scheduling policies:

- FIFO
- Priority scheduling
- Cost-aware scheduling
- Deadline scheduling
- Dependency-aware scheduling
- Round-robin

Future ideas: GPU-aware scheduling, Token-budget scheduling, Energy-aware scheduling

Event Bus

Agents never directly invoke other agents. Instead they publish events.

Example:

Research finishes

↓

Publishes ResearchCompleted

The runtime determines who subscribed.

ResearchCompleted

↓

Wake CodeAgent

↓

Wake DocumentationAgent

↓

Notify Planner

Supported event types:

- AgentFinished
- AgentFailed
- ToolCompleted
- HumanApproved
- MemoryUpdated
- ModelFinished
- TimerExpired
- FileCreated

This produces loosely coupled agent systems.

Dependency Graph

One of the most important features. Instead of static workflows, AgentOS maintains a dependency graph.

Example:

Research depends on: Market Search, Legal Review, Human Approval

Once every dependency completes, the scheduler automatically wakes the waiting process.

Dependencies can include: other agents, tools, timers, events, human approvals, external APIs

Human Approval as a First-Class System Object

Instead of confidence thresholds, agents explicitly declare approval requirements.

Example:

- Need Approval
- Role: Senior Engineer
- Reason: Production deployment

The runtime blocks the process. Status becomes: BLOCKED. Waiting On: Senior Engineer

After approval, the scheduler resumes execution. This treats humans as system resources rather than tools.

Memory Manager

Instead of every framework implementing memory differently, AgentOS provides memory primitives.

Types of memory:

- Working Memory: current reasoning
- Scratchpad: temporary notes
- Shared Memory: accessible by multiple agents
- Long-Term Memory: persistent knowledge
- Semantic Memory: vector storage
- Episodic Memory: execution history

Memory API:

- `memory.store()`
- `memory.retrieve()`
- `memory.share()`
- `memory.delete()`

Implementation could use: SQLite, Redis, Vector database, Filesystem, RAM cache. Applications remain unaware of the backend.

Tool Driver System

Tools behave like operating system device drivers. Instead of importing GitHub libraries directly, agents request capabilities.

Example:

Need: GitHub

The runtime selects the correct driver. Possible drivers:

- Python
- Browser
- SQL
- GitHub
- Filesystem
- Docker
- Shell

- Email
- Calendar
- Slack
- REST APIs
- Future MCP servers

Driver responsibilities: authentication, permissions, rate limiting, retry logic, logging, caching, error handling

Permission System

Every process receives capabilities.

Example:

- Research Agent: ✓ Browser, ✓ Search, ✗ SQL, ✗ Shell
- Coder: ✓ Python, ✓ GitHub, ✓ Docker
- Finance: ✓ SQL, ✓ Email, ✗ Browser

Kernel validates permissions before every tool execution. Possible future additions: role inheritance, temporary permissions, approval-based permissions

Model Manager

Agents never choose models directly. Instead they request capabilities.

Example:

Need: Large reasoning model

The runtime selects: GPT-5, Claude, Gemini, Local model, Reasoning model, Code model, Vision model

Selection criteria: cost, latency, context size, availability, GPU memory, quality

Future: automatic model benchmarking, dynamic model switching

Checkpoint Manager

Running agents periodically save state. Checkpoint includes:

- reasoning
- memory
- tool outputs
- current task

- dependency graph
- event subscriptions

If interrupted, the runtime restores from the last checkpoint. Applications no longer restart entire workflows.

Agent Tree

Agents can spawn child agents.

```
Example:
Planner
├─ Research A
├─ Research B
├─ Research C
└─ Documentation
```

Parent agents: create children, wait for children, terminate children, aggregate results

Shared Runtime

Unlike normal Python libraries, AgentOS exists independently of applications. Multiple applications connect to the same runtime.

Application A, Application B, Application C

↓

AgentOS

The runtime maintains global knowledge of: all agents, all models, all tool usage, all GPU utilization, all costs, all events. This enables global optimization.

Monitoring

Provide both CLI and Web UI.

CLI: agent ps, agent top, agent logs, agent kill, agent wait, agent events

Dashboard: Running Agents, Waiting Agents, Blocked Agents, Dependency Graph, Memory Usage, GPU Usage, Model Usage, Tool Usage, Event Timeline, Cost, Latency

Logging

Every event is recorded. **Example:**

- 12:30 - Planner Spawned

- 12:31 - Research Started
- 12:33 - Browser Search
- 12:34 - Checkpoint Saved
- 12:35 - Human Approval Requested
- 12:40 - Approved
- 12:41 - Execution Resumed

APIs

Example API:

- `kernel.spawn()`
- `kernel.kill()`
- `kernel.pause()`
- `kernel.resume()`
- `kernel.wait()`
- `kernel.publish()`
- `kernel.subscribe()`
- `kernel.memory.store()`
- `kernel.memory.retrieve()`
- `kernel.request_tool()`
- `kernel.request_model()`
- `kernel.checkpoint()`
- `kernel.logs()`

Suggested Tech Stack

- Language: Python
- Backend: FastAPI
- Database: SQLite initially, Later PostgreSQL, Optional Redis
- Vector Database: FAISS or Chroma
- Async Runtime: asyncio
- Visualization: React, D3.js or Cytoscape
- LLM Routing: LiteLLM
- Monitoring: WebSockets
- Docker deployment

Folder Structure

```
agentos/  
  kernel/  
    scheduler.py  
    process.py  
    memory.py  
    permissions.py  
    events.py  
    models.py  
    checkpoints.py  
    tools.py  
  runtime/  
    daemon.py  
    executor.py  
  agents/  
    planner.py  
    research.py  
    coder.py  
    reviewer.py  
  drivers/  
    browser.py  
    python.py  
    github.py  
    sql.py  
    shell.py  
  api/  
    FastAPI  
  dashboard/  
    React UI  
  examples/  
    software_company/  
    research_assistant/  
    customer_support/
```

Long-Term Vision

AgentOS should become a standard execution platform for autonomous AI agents, analogous to how Linux is the standard execution platform for traditional software.

Its purpose is not to replace Linux, but to provide a consistent, operating system-inspired abstraction for intelligent computation.

Instead of every AI application reinventing orchestration, memory, permissions, scheduling, checkpoints, and tool integration, developers build against AgentOS's APIs.

The runtime becomes the central authority responsible for agent lifecycle, communication, resource allocation, dependency resolution, and recovery.

Research Contributions / Novel Ideas

The project's novelty should come from OS-inspired abstractions for AI, not from wrapping existing agent frameworks. Potential contributions include:

- Agents as first-class processes with lifecycle states, PIDs, parent-child relationships, and managed execution.
- Dependency-aware scheduling, where agents are resumed automatically when required agents, tools, timers, or approvals complete, rather than following predefined workflows.
- Human approvals as kernel objects, allowing humans to participate in dependency graphs instead of acting as ad hoc fallbacks.
- Capability-based tool access, where agents request capabilities and the runtime enforces permissions independently of the application.
- Centralized model routing and resource management, enabling cost, latency, and hardware-aware optimization across multiple applications.
- A shared runtime that can coordinate many independent AI applications, providing global visibility into agents, memory, models, events, and costs.
- The overarching goal is to shift AI development from application-managed orchestration to runtime-managed intelligent execution, much as operating systems shifted software development from manually managing hardware to programming against standardized system abstractions.

Implementation Roadmap

This section was appended to the original design document to make it self-continuing. Everything above (pages 1–11) is the specification. Everything below is the build plan: eight phases, each one implementable end to end and verifiable by a human in under two minutes. An agent picking this document up later should read the phase table, find the first phase not marked Done, and implement it against the verification criteria stated there.

Governing principle

The kernel must be demonstrable with fake agents before a single LLM call is made.

Dependency-aware scheduling, human approval as a kernel object, and crash recovery are the research contributions of this project (p.11). If they can only be observed through a nondeterministic LLM, they cannot be verified, debugged, or defended. Every agent in the examples and tests up to Phase 5 does nothing but sleep and return a value. Scheduling bugs therefore reproduce deterministically, at zero token cost, with no API key present.

Architectural decision: the process model

Agents are **asyncio tasks behind a strict message-passing boundary** — the hybrid of the three options considered (plain coroutines; real OS subprocesses from day one; hybrid). An agent never holds a reference to the kernel, the process table, or another agent. Its entire world is a Context object exposing spawn/sleep/wait/log, and every one of those calls crosses an asyncio queue as a JSON-serializable Syscall and returns as a Reply. Serializability is asserted at runtime, not assumed.

This is the load-bearing decision of the whole project. Because a payload that survives json.dumps can survive a pipe, Phase 7 can relocate agents into real OS subprocesses by changing only the Executor — agents/, kernel/, and every example remain untouched. Taking the shortcut (letting agents share memory with the kernel) would make 'agents are processes' a metaphor rather than an enforced boundary, and retrofitting isolation later would mean a rewrite.

Phase 1 — Process table and lifecycle

Status: DONE (v0.1.0, stdlib only)

Build

- kernel/states.py — the 9 lifecycle states from p.3 with an *enforced* transition table; an illegal transition raises InvalidTransition rather than silently corrupting the process table.
- kernel/process.py — the process table and PID allocation; every agent carries the p.3 metadata block (PID, name, parent, children, status, priority, waiting-on).
- kernel/messages.py — the Syscall/Reply boundary and the serializability assertion.
- kernel/scheduler.py — FIFO, behind a policy interface that takes the whole ready queue so Phase 2 can add more policies without reshaping the kernel loop.
- kernel/kernel.py — the kernel loop: spawn, kill (cascades to descendants, never to ancestors), pause, resume, wait. Slot admission: an agent runs only when the scheduler grants it a slot.
- kernel/store.py — SQLite at .agentos/runtime.db. The kernel publishes the process table; the CLI reads it and pushes control commands back. Deliberately the same control-plane shape as the Phase 7 daemon, so that swap is a transport change and not a redesign.
- runtime/executor.py — one asyncio task per agent, plus the Context proxy.
- cli.py — agent ps / top / logs / kill / pause / resume / run (p.8).
- examples/tree.py — the Planner + 3 Research + Documentation tree from p.8. No LLM.

Verify (the human runs these)

- python -m unittest discover tests -v — 13 tests pass.
- python -m agentos.cli run examples/tree.py --slots 2 — five agents, two slots, so the tree is forced to queue; the Planner aggregates all three research results.
- From a second terminal while it runs: agent ps shows the live tree and each agent's status and waiting-on; agent kill 3 kills a Research child and the Planner keeps running; agent logs prints every state transition.

Done when: the state transitions in agent logs match the lifecycle diagram on p.3, and a woken agent is observed going Sleeping → Ready → Running (re-queuing for a slot) rather than resuming instantly. That distinction is what makes it a scheduler rather than a callback, and a test locks it in.

Phase 2 — Event bus, dependency graph, real scheduling

Status: DONE (v0.2.0, still stdlib only)

Build

- kernel/events.py — publish/subscribe over the 8 kernel event types from p.5, plus application-defined ones (the doc's own ResearchCompleted is one of those). Delivery is *buffered per subscriber*, not broadcast-and-forget: an event that fires while a subscriber is busy still lands in its queue. Without that buffer, 'did I subscribe before you published?' is a race, and races in a scheduler are the bugs you never reproduce.

- `kernel/depgraph.py` — the dependency graph. `ctx.wait_all(agents=[...], events=[...], timer=...)` declares a dependency set; the scheduler wakes the agent automatically once the last one resolves. Nobody wrote a sequence — the graph decided the order.
- Deadlock detection, two kinds. (a) A wait that would close a cycle in the wait-for graph is refused at the moment it is requested, and the agent gets an error it can name. (b) A stall with no cycle — every surviving agent Waiting, nobody Sleeping, no timer pending — means nothing can ever happen again, and is reported rather than hung on.
- Scheduling policies (p.4): Priority (with an ageing guard, because a priority scheduler that cannot starve anyone is not a priority scheduler) and `DependencyAware` — run whoever unblocks the most work, which is the policy a CPU scheduler cannot have, since only this kernel knows that six agents are blocked on pid 12.
- `agents/base.py` — 'agents never directly invoke other agents' (p.5) is now *enforced*: `Agent.run` is wrapped so it can only be entered by the executor. Calling another agent's `run()` directly raises `DirectInvocationError`. Left as a convention, the first person in a hurry quietly turns AgentOS back into a function-call framework.
- `cli.py` — agent events, the p.5 timeline: which event fired, who published it, whom it woke.
- `examples/pipeline.py` (the p.5 scenario) and `examples/deadlock.py` (both stall modes).

Verify (the human runs these)

- `python -m unittest discover tests -v` — 28 tests pass (13 from Phase 1, 15 new).
- `python -m agentos.cli run examples/pipeline.py` — Research publishes `ResearchCompleted` and the runtime wakes `CodeAgent`, `DocumentationAgent`, and `Reviewer`. Read `Research.run()`: it never mentions any of them.
- `python -m agentos.cli events -v` — the timeline shows one publish waking three subscribers.
- `python -m agentos.cli run examples/deadlock.py` — a wait cycle is refused, and an unsatisfiable event wait is detected. Neither run hangs.

Done when: a new subscriber can be added without editing a single existing line of any other agent. `examples/pipeline.py` demonstrates this: `Reviewer` is a fourth subscriber, and adding it required appending one spawn and touching nothing else.

Phase 3 — Human approval as a first-class kernel object

Status: TODO — next

Build

- `kernel.request_approval(role=..., reason=...)` blocks the process, sets status BLOCKED with waiting-on the named role, and registers the human as a node in the dependency graph — identical in kind to a tool, a timer, or another agent (p.5–6).
- `agent approve <pid> --as <role>` to satisfy the dependency; the scheduler then resumes the process normally.

Verify (the human runs these)

- Run the deploy example. It stops. `agent ps` shows BLOCKED, waiting on Senior Engineer. Approving it resumes execution from exactly where it stopped.
- Kill the runtime while an agent is blocked, restart, and confirm the block survives.

Done when: the pending approval survives a runtime restart. A human dependency that evaporates on restart is not a kernel object — it is a callback with good PR. This phase is the project's most defensible novel claim (p.11), which is why it lands before tools, memory, and models rather than after them.

Phase 4 — Permissions and tool drivers

Status: TODO

Build

- `kernel/permissions.py` — capability-based access (p.7). The kernel validates capabilities *before* dispatch, independently of the application.
- `drivers/` — python, shell, filesystem, browser, REST, SQL. Each driver owns authentication, rate limiting, retries, logging, and error handling (p.6–7).
- Agents request a capability ('Need: GitHub'); they never import a library directly.

Verify (the human runs these)

- The Finance agent's SQL call succeeds and its Browser call is denied by the kernel, with the denial in the audit log (p.7 permission matrix).
- Revoking a capability in config changes agent behaviour with no code edit.

Done when: an agent cannot reach a tool it was not granted, even if it tries to import the underlying library itself.

Phase 5 — Memory manager, then model routing

Status: TODO

Build

- `kernel/memory.py` — all six memory types (p.6) behind store/retrieve/share/delete. SQLite plus a vector store; the backend stays invisible to the agent.

- kernel/models.py — kernel.request_model() via LiteLLM (p.7). Agents request a capability ('Need: large reasoning model'), never a model name. The runtime selects on cost, latency, context size, and availability, and records tokens and cost per agent.

Verify (the human runs these)

- Two agents share state through the kernel's memory API and never touch each other.
- The first real LLM run. Swapping the routing config from a frontier model to a local one leaves agent code untouched.
- agent top shows cost accruing per agent.

Done when: model choice is a runtime configuration concern and not an application concern. **This is the first phase that requires an API key** — everything before it is verifiable offline.

Phase 6 — Checkpoints and crash recovery

Status: TODO

Build

- kernel/checkpoints.py — periodic snapshots of reasoning, memory, tool outputs, current task, dependency graph, and event subscriptions (p.7–8). Restore on restart.
- This is where the Phase 1 spec discipline pays off: an agent is re-creatable from its spec (module, qualname, params), which is why agent constructor params are required to be serializable from day one.

Verify (the human runs these)

- Start a multi-agent run, kill -9 the runtime mid-execution, restart it, and watch agents resume from their last checkpoint instead of re-running from scratch.

Done when: a hard kill costs the work since the last checkpoint and nothing more. This is the demo that sells the project — no mainstream agent framework does it well.

Phase 7 — Shared runtime daemon

Status: TODO

Build

- runtime/daemon.py plus api/ (FastAPI) — the runtime becomes a process that outlives any application (p.8).
- A thin client: applications connect to a runtime that already exists instead of instantiating one.
- Swap the Executor's asyncio tasks for real OS subprocesses. Because of the Phase 1 message boundary, agents/ and kernel/ do not change.

Verify (the human runs these)

- Two separate applications in two terminals; one agent_t ps shows agents from both, with cost and model usage aggregated across them.

Done when: global visibility across independent applications is real (p.8). This is the claim that separates AgentOS from a library.

Phase 8 — Dashboard, examples, and the evaluation

Status: TODO

Build

- dashboard/ — React plus WebSockets: running/waiting/blocked agents, live dependency graph, event timeline, cost, latency (p.8).
- examples/ — software_company, research_assistant, customer_support (p.10).
- **A benchmark against LangGraph or CrewAI** on recovery-after-crash, human-in-the-loop latency, and cost under multi-application load.

Verify (the human runs these)

- The dashboard can be shown to someone live, and the benchmark produces a table of numbers.

Done when: the novelty claims on p.11 are backed by measurements rather than assertions. The benchmark is what turns this from a project into research; the dashboard is the most tempting and least load-bearing piece, which is exactly why it is last.

Notes for whoever continues this

- Phases 1–3 *are* the project. Phases 4–8 are execution. If dependency-aware scheduling with human approvals as graph nodes works, the rest is engineering that can be finished.
- Do not build the dashboard early. It demos well and proves nothing.
- Keep the fake-agent examples working forever. When an LLM-backed agent misbehaves in Phase 5+, the sleep-only examples are how you tell a kernel bug from a model bug.
- The serializability assertion at the message boundary is not ceremony. Every time it fires, it is preventing a Phase 6 or Phase 7 bug that would otherwise surface months later as an unpicklable checkpoint.
- Run `python -m unittest discover tests -v` before and after every phase. Phase 1 ships 13 tests; a phase that adds no tests has not been verified.